

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 2

За седьмой семестр

Тема: «Многослойный персептрон для задачи «исключающее ИЛИ» с числовыми классами: сравнение функций потерь MSE и Binary Cross-Entropy»

Выполнил:
Студент 4 курса
Группы ИИ-26
Згера Е. А.
Проверил:
Никитин С.К.

Вариант 5

Цель: На основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами c_0 и c_1 из диапазона $[-10; 10]$, реализовать альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провести сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.

Задачи лабораторной работы

1. Взять за основу код многослойного персептрона 2-2-1 из Лабораторной работы №1.
2. Поскольку BCE определена для выходов в диапазоне $(0; 1)$, а целевые значения в вашем варианте – произвольные числа c_0 и c_1 из диапазона $[-10; 10]$, реализовать нормализацию (масштабирование) целевых значений перед обучением и обратное преобразование выхода сети после обучения:
 - привести c_0 и c_1 к бинарным меткам 0 и 1;
 - обучать сеть на нормализованных целевых значениях с функцией потерь BCE;
 - при выводе результата преобразовывать выход сети $\hat{y} \in (0; 1)$ обратно в шкалу $[c_0; c_1]$.
3. Реализовать корректный градиент для связки BCE + сигмоида на выходном слое; для скрытого слоя градиент передаётся по стандартному правилу цепочки.
4. Обучить сеть на полной таблице истинности своего варианта (4 примера) в двух конфигурациях:
 - **Конфигурация А (базовая, из ЛР №1):** MLP 2-2-1, функция потерь MSE, целевые значения – исходные c_0/c_1 (без нормализации), выходной слой – сигмоида. Результаты можно взять из отчёта по ЛР №1 или, при необходимости, повторить обучение с фиксацией всех метрик.
 - **Конфигурация Б (новая):** MLP 2-2-1, функция потерь BCE, целевые значения – нормализованные $(0/1)$, с последующим пересчётом выхода в шкалу $[c_0; c_1]$.

Критерий остановки для обеих конфигураций одинаков – суммарная ошибка $E_s \leq E_e$ (используйте то же значение E_e , что и в ЛР №1, либо согласуйте новое значение отдельно для MSE и для BCE, поскольку это разные по своей природе величины – обоснуйте выбор в отчёте).

5. Для каждой конфигурации зафиксировать:
 - количество эпох до достижения критерия остановки (либо факт недостижения за максимально допустимое число эпох);
 - итоговую суммарную ошибку;
 - ассигасу на всех 4 примерах таблицы истинности;
 - среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.
6. Провести не менее 5 запусков обучения для каждой конфигурации с разной случайной инициализацией весов (менять seed) и сравнить устойчивость сходимости: в скольких из 5 запусков сеть достигла критерия остановки, и какой разброс по числу эпох наблюдается.
7. Построить общий график зависимости суммарной ошибки от номера эпохи для конфигурации А (MSE) и конфигурации Б (BCE) на одних осях (для представительного запуска каждой конфигурации).
8. Реализовать блок визуализации результатов, включающий:

- **График сходимости** – суммарная ошибка E_s от номера эпохи, конфигурации А и Б на одних осях, разными цветами/стилями линий, с обозначением порога E_e .
- **Диаграмму разброса числа эпох по 5 запускам** (bar chart или scatter) для конфигурации А и конфигурации Б – наглядно показывающую устойчивость сходимости из п. 6 (например, столбики числа эпох для каждого из 5 seed, сгруппированные по конфигурации; запуски, не достигшие критерия останова, отметить отдельно, например штриховкой или отдельным цветом).
- **Визуализацию разделяющей поверхности скрытого слоя** – для представительного обученного экземпляра каждой конфигурации построить на плоскости (А, В) карту выходного значения сети (heatmap или contour plot) с наложенными точками обучающей выборки (4 примера таблицы истинности), закрашенными по истинному классу. Диапазон осей А и В – $[-10; 10]$ (весь допустимый диапазон входных значений). Для конфигурации Б цветовую шкалу можно строить как по нормализованному $\hat{y} \in (0; 1)$, так и по пересчитанному значению в шкале $[c_0; c_1]$ – выбрать более наглядный вариант и обосновать выбор.
- **Диаграмму сравнения точности восстановления шкалы** – столбчатую диаграмму средней абсолютной ошибки для конфигурации А и конфигурации Б, чтобы наглядно сопоставить точность регрессии к исходной шкале $[c_0; c_1]$.

Каждый график должен иметь подписанные оси, заголовок и легенду (где применимо).

Разделяющую поверхность и графики точности рекомендуется разместить рядом (например, subplot 2×2 или отдельными рисунками) для удобства сравнения в отчёте.

9. Реализовать режим функционирования сети: пользователь вводит пару чисел (А, В) из диапазона $[-10; 10]$ (не обязательно совпадающих с c_0 и c_1). Сеть должна выводить:

- выход в нормализованной шкале $\hat{y} \in (0; 1)$ (с точностью до 4 знаков);
- выход, пересчитанный в исходную шкалу $[c_0; c_1]$ по формуле из п. 2;
- к какому классу (c_0 или c_1) ближе полученное значение.

Продемонстрировать работу режима функционирования на всех 4 примерах исходной таблицы истинности и не менее чем на 2–3 дополнительных парах (А, В), не входящих в обучающую выборку.

Вариант 5 ($c_0 = 1, c_1 = 8$)

А	В	$A \oplus B$
1	1	1
1	8	8
8	1	8
8	8	1

Описание изменений в архитектуре и алгоритме обучения:

Архитектура сети не изменилась: MLP 2-2-1 (два входных нейрона, два скрытых, один выходной), сигмоида на обоих слоях.

Изменения касаются исключительно **способа подготовки целевых значений и функции потерь**.

2.1. Нормализация целевых значений

В ЛР №1 целевые значения нормализовались линейно (min-max scaling) в диапазон (0;1):

В ЛР №2 (конфигурация Б) вместо линейного масштабирования исходные классы приводятся к **бинарным меткам**, что соответствует смыслу ВСЕ (задача бинарной классификации, а не регрессии):

Для варианта $c_0 = 1, c_1 = 8$: $(1,1) \rightarrow 0, (1,8) \rightarrow 1, (8,1) \rightarrow 1, (8,8) \rightarrow 0$.

2.2. Обратный пересчёт выхода

После обучения выход сети $\hat{y} \in (0;1)$ пересчитывается обратно в исходную шкалу $[c_0; c_1]$ по формуле линейной интерполяции:

Это та же по форме операция, что обратное масштабирование в ЛР №1, но применяется не к MSE-регрессии, а к вероятности принадлежности классу c_1 , выданной сетью.

2.3. Функция потерь ВСЕ

Вместо MSE используется бинарная кросс-энтропия

Градиент для скрытого слоя вычисляется по обычному правилу цепочки, без изменений относительно ЛР №1

Код программы

```
# -*- coding: utf-8 -*-

import numpy as np
import matplotlib.pyplot as plt

np.set_printoptions(precision=4, suppress=True)

c0, c1 = 1, 8

X = np.array([[c0, c0], [c0, c1], [c1, c0], [c1, c1]], dtype=float)
y_raw = np.array([[c0], [c1], [c1], [c0]], dtype=float)
y_bin = np.array([[0.], [1.], [1.], [0.]]) # бинарные метки для ВСЕ

LR = 0.1
MAX_EPOCHS = 20000
EE_MSE = 0.01
EE_BCE = 0.35
SEEDS = [0, 1, 2, 3, 4]

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-np.clip(x, -500, 500)))

def sigmoid_derivative_from_output(o):
    return o * (1.0 - o)

def init_weights(seed, n_in=2, n_hidden=2, n_out=1):
    rng = np.random.RandomState(seed)
    W1 = rng.uniform(-0.5, 0.5, (n_in, n_hidden))
    b1 = np.zeros((1, n_hidden))
    W2 = rng.uniform(-0.5, 0.5, (n_hidden, n_out))
    b2 = np.zeros((1, n_out))
    return W1, b1, W2, b2

def forward(Xb, W1, b1, W2, b2):
    h_out = sigmoid(Xb.dot(W1) + b1)
    o_out = sigmoid(h_out.dot(W2) + b2)
    return h_out, o_out

def train_mse(seed, lr=LR, max_epochs=MAX_EPOCHS, Ee=EE_MSE):
```

```

W1, b1, W2, b2 = init_weights(seed)
errors = []
converged_epoch = None

for epoch in range(max_epochs):
    h_out, o_out = forward(X, W1, b1, W2, b2)

    error = y_raw - o_out
    Es = np.mean(error ** 2)
    errors.append(Es)

    if Es <= Ee:
        converged_epoch = epoch
        break

    d_output = error * sigmoid_derivative_from_output(o_out)
    d_hidden = d_output.dot(W2.T) * sigmoid_derivative_from_output(h_out)

    W2 += h_out.T.dot(d_output) * lr
    b2 += np.sum(d_output, axis=0, keepdims=True) * lr
    W1 += X.T.dot(d_hidden) * lr
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * lr

return dict(W1=W1, b1=b1, W2=W2, b2=b2, errors=errors, converged_epoch=converged_epoch)

def train_bce(seed, lr=LR, max_epochs=MAX_EPOCHS, Ee=EE_BCE, eps=1e-9):
    W1, b1, W2, b2 = init_weights(seed)
    errors = []
    converged_epoch = None
    n = X.shape[0]

    for epoch in range(max_epochs):
        h_out, o_out = forward(X, W1, b1, W2, b2)

        o_clip = np.clip(o_out, eps, 1 - eps)
        Es = -np.mean(y_bin * np.log(o_clip) + (1 - y_bin) * np.log(1 - o_clip))
        errors.append(Es)

        if Es <= Ee:
            converged_epoch = epoch
            break

        # градиент BCE+сигмоида: (o_out - target)
        d_output = (o_out - y_bin) / n
        d_hidden = d_output.dot(W2.T) * sigmoid_derivative_from_output(h_out)

        W2 -= h_out.T.dot(d_output) * lr
        b2 -= np.sum(d_output, axis=0, keepdims=True) * lr
        W1 -= X.T.dot(d_hidden) * lr
        b1 -= np.sum(d_hidden, axis=0, keepdims=True) * lr

    return dict(W1=W1, b1=b1, W2=W2, b2=b2, errors=errors, converged_epoch=converged_epoch)

def nearest_class(value, c0, c1):
    return c0 if abs(value - c0) <= abs(value - c1) else c1

def rescale_bce_output(o_out, c0, c1):
    return c0 + o_out * (c1 - c0)

def evaluate_run(result, config):
    h_out, o_out = forward(X, result["W1"], result["b1"], result["W2"], result["b2"])

    if config == "A":
        pred_scale = o_out.flatten()
    else:
        pred_scale = rescale_bce_output(o_out.flatten(), c0, c1)

    true_scale = y_raw.flatten()
    pred_class = np.array([nearest_class(v, c0, c1) for v in pred_scale])
    accuracy = np.mean(pred_class == true_scale)
    mae = np.mean(np.abs(pred_scale - true_scale))
    return accuracy, mae, pred_scale

def run_experiments():
    results_A, results_B = [], []

    print(f"КОНФИГУРАЦИЯ A (MSE, без нормализации, Ee = {EE_MSE})")

```

```

for seed in SEEDS:
    res = train_mse(seed)
    acc, mae, _ = evaluate_run(res, "A")
    res.update(seed=seed, accuracy=acc, mae=mae)
    results_A.append(res)
    status = f"Эпоха {res['converged_epoch']}" if res["converged_epoch"] is not None \
        else f"НЕ СОШЛАСЬ за {MAX_EPOCHS} эпох"
    print(f"seed={seed}: {status}, Es_final={res['errors'][-1]:.4f}, accuracy={acc:.2f}, MAE={mae:.2f}")

print(f"\nКОНФИГУРАЦИЯ Б (BCE, нормализованные метки 0/1, Ee = {EE_BCE})")
for seed in SEEDS:
    res = train_bce(seed)
    acc, mae, _ = evaluate_run(res, "B")
    res.update(seed=seed, accuracy=acc, mae=mae)
    results_B.append(res)
    status = f"Эпоха {res['converged_epoch']}" if res["converged_epoch"] is not None \
        else f"НЕ СОШЛАСЬ за {MAX_EPOCHS} эпох"
    print(f"seed={seed}: {status}, Es_final={res['errors'][-1]:.4f}, accuracy={acc:.2f}, MAE={mae:.2f}")

return results_A, results_B

def convergence_summary(results, label):
    converged = [r for r in results if r["converged_epoch"] is not None]
    epochs = [r["converged_epoch"] for r in converged]
    print(f"\n{label}: сошлось {len(converged)} из {len(results)}")
    if epochs:
        print(f"Эпоха до сходимости: min={min(epochs)}, max={max(epochs)}, "
              f"среднее={np.mean(epochs):.1f}, разброс={max(epochs) - min(epochs)}")
    else:
        print("Ни один запуск не достиг критерия остановки.")

def plot_convergence(results_A, results_B, rep_seed=0):
    rep_A = next(r for r in results_A if r["seed"] == rep_seed)
    rep_B = next(r for r in results_B if r["seed"] == rep_seed)

    fig, ax1 = plt.subplots(figsize=(8, 5))
    ax1.plot(rep_A["errors"], color="tab:blue", label="Конфигурация А (MSE)")
    ax1.axhline(EE_MSE, color="tab:blue", linestyle="--", alpha=0.5, label=f"Ee(MSE)={EE_MSE}")
    ax1.set_xlabel("Эпоха")
    ax1.set_ylabel("Es (MSE)", color="tab:blue")
    ax1.tick_params(axis="y", labelcolor="tab:blue")

    ax2 = ax1.twinx()
    ax2.plot(rep_B["errors"], color="tab:orange", label="Конфигурация Б (BCE)")
    ax2.axhline(EE_BCE, color="tab:orange", linestyle="--", alpha=0.5, label=f"Ee(BCE)={EE_BCE}")
    ax2.set_ylabel("Es (BCE)", color="tab:orange")
    ax2.tick_params(axis="y", labelcolor="tab:orange")

    lines1, labels1 = ax1.get_legend_handles_labels()
    lines2, labels2 = ax2.get_legend_handles_labels()
    ax1.legend(lines1 + lines2, labels1 + labels2, loc="upper right")

    plt.title("Сходимость: Конфигурация А (MSE) и Конфигурация Б (BCE)")
    fig.tight_layout()
    plt.savefig("plot_1_convergence.png", dpi=150)
    plt.show()

def plot_epochs_bar(results_A, results_B):
    fig, ax = plt.subplots(figsize=(9, 5))
    x = np.arange(len(SEEDS))
    width = 0.35

    epochs_A = [r["converged_epoch"] if r["converged_epoch"] is not None else MAX_EPOCHS for r in results_A]
    epochs_B = [r["converged_epoch"] if r["converged_epoch"] is not None else MAX_EPOCHS for r in results_B]
    conv_A = [r["converged_epoch"] is not None for r in results_A]
    conv_B = [r["converged_epoch"] is not None for r in results_B]

    bars_A = ax.bar(x - width / 2, epochs_A, width, label="Конфигурация А (MSE)", color="tab:blue")
    bars_B = ax.bar(x + width / 2, epochs_B, width, label="Конфигурация Б (BCE)", color="tab:orange")

    for bar, ok in zip(bars_A, conv_A):
        if not ok:
            bar.set_hatch("/")
            bar.set_edgecolor("red")
    for bar, ok in zip(bars_B, conv_B):
        if not ok:
            bar.set_hatch("/")
            bar.set_edgecolor("red")

```

```

ax.set_xticks(x)
ax.set_xticklabels([f"seed={s}" for s in SEEDS])
ax.set_ylabel("Эпох до сходимости")
ax.set_title("Устойчивость сходимости по 5 запускам (штриховка = не сошлось)")
ax.legend()
fig.tight_layout()
plt.savefig("plot_2_epochs_bar.png", dpi=150)
plt.show()

def plot_decision_surfaces(results_A, results_B, rep_seed=0):
    rep_A = next(r for r in results_A if r["seed"] == rep_seed)
    rep_B = next(r for r in results_B if r["seed"] == rep_seed)

    a_range = np.linspace(-10, 10, 200)
    b_range = np.linspace(-10, 10, 200)
    AA, BB = np.meshgrid(a_range, b_range)
    grid = np.column_stack([AA.ravel(), BB.ravel()])

    fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))

    _, o_A = forward(grid, rep_A["W1"], rep_A["b1"], rep_A["W2"], rep_A["b2"])
    ZZ_A = o_A.reshape(AA.shape)
    cs0 = axes[0].contourf(AA, BB, ZZ_A, levels=20, cmap="coolwarm")
    fig.colorbar(cs0, ax=axes[0], label="Выход сети")
    axes[0].scatter(X[:, 0], X[:, 1], c=y_raw.flatten(), cmap="coolwarm", edgecolors="black", s=120,
linewidths=1.5)
    axes[0].set_title("Конфигурация А (MSE)")
    axes[0].set_xlabel("A")
    axes[0].set_ylabel("B")

    _, o_B = forward(grid, rep_B["W1"], rep_B["b1"], rep_B["W2"], rep_B["b2"])
    ZZ_B = o_B.reshape(AA.shape)
    cs1 = axes[1].contourf(AA, BB, ZZ_B, levels=20, cmap="coolwarm")
    fig.colorbar(cs1, ax=axes[1], label="ŷ (0..1)")
    axes[1].contour(AA, BB, ZZ_B, levels=[0.5], colors="black", linewidths=2)
    axes[1].scatter(X[:, 0], X[:, 1], c=y_bin.flatten(), cmap="coolwarm", edgecolors="black", s=120,
linewidths=1.5)
    axes[1].set_title("Конфигурация Б (BCE)")
    axes[1].set_xlabel("A")
    axes[1].set_ylabel("B")

    fig.suptitle("Разделяющая поверхность скрытого слоя (A,B ∈ [-10;10])")
    fig.tight_layout()
    plt.savefig("plot_3_decision_surfaces.png", dpi=150)
    plt.show()

def plot_mae_comparison(results_A, results_B):
    mae_A = np.mean([r["mae"] for r in results_A])
    mae_B = np.mean([r["mae"] for r in results_B])

    fig, ax = plt.subplots(figsize=(6, 5))
    bars = ax.bar(["Конфигурация А (MSE)", "Конфигурация Б (BCE)"], [mae_A, mae_B], color=["tab:blue",
"tab:orange"])
    for bar, val in zip(bars, [mae_A, mae_B]):
        ax.text(bar.get_x() + bar.get_width() / 2, val, f"{val:.2f}", ha="center", va="bottom")
    ax.set_ylabel(f"Средняя МАЕ в шкале [{c0}; {c1}]")
    ax.set_title("Точность восстановления исходной шкалы")
    fig.tight_layout()
    plt.savefig("plot_4_mae_comparison.png", dpi=150)
    plt.show()

def predict_B(a, b, result):
    h_out, o_out = forward(np.array([[a, b]], dtype=float), result["W1"], result["b1"], result["W2"],
result["b2"])
    y_hat_norm = o_out.item()
    y_hat_scaled = rescale_bce_output(y_hat_norm, c0, c1)
    y_class = nearest_class(y_hat_scaled, c0, c1)
    return y_hat_norm, y_hat_scaled, y_class

def demo_predictions(result):
    print("\nРЕЖИМ ФУНКЦИОНИРОВАНИЯ СЕТИ (конфигурация Б)")

    print("\n--- Все 4 примера обучающей таблицы ---")
    for a, b in X:
        y_hat_norm, y_hat_scaled, y_class = predict_B(a, b, result)
        print(f"Вход:  ({a:.0f}, {b:.0f})  ->  ŷ_norm={y_hat_norm:.4f},  ŷ_scaled={y_hat_scaled:.2f},
класс≈{y_class}")

```

```

print("\n--- Дополнительные пары (вне обучающей выборки) ---")
extra_pairs = [(3, 3), (2, 7), (5, 5), (-4, 6)]
for a, b in extra_pairs:
    y_hat_norm, y_hat_scaled, y_class = predict_B(a, b, result)
    print(f"Вход: ({a}, {b}) ->  $\hat{y}_{norm}={y\_hat\_norm:.4f}$ ,  $\hat{y}_{scaled}={y\_hat\_scaled:.2f}$ , класс={y_class}")

def interactive_mode(result):
    print("\n--- Интерактивный режим (введите 'q' для выхода) ---")
    while True:
        raw = input("Введите A,B через пробел или запятую в диапазоне [-10;10]: ").strip()
        if raw.lower() == "q":
            break
        try:
            a_str, b_str = raw.replace(", ", " ").split()
            a, b = float(a_str), float(b_str)
        except ValueError:
            print("Не удалось разобрать ввод, попробуйте снова (пример: 3 7).")
            continue
        y_hat_norm, y_hat_scaled, y_class = predict_B(a, b, result)
        print(f" $\hat{y}_{norm}={y\_hat\_norm:.4f}$  |  $\hat{y}$  в шкале [{c0};{c1}]= $\hat{y}_{scaled:.2f}$  | ближе к классу {y_class}")

if __name__ == "__main__":
    results_A, results_B = run_experiments()

    convergence_summary(results_A, "Конфигурация А (MSE)")
    convergence_summary(results_B, "Конфигурация Б (BCE)")

    plot_convergence(results_A, results_B, rep_seed=0)
    plot_epochs_bar(results_A, results_B)
    plot_decision_surfaces(results_A, results_B, rep_seed=0)
    plot_mae_comparison(results_A, results_B)

    rep_B = next(r for r in results_B if r["seed"] == 0)
    demo_predictions(rep_B)

```

Таблица результатов:

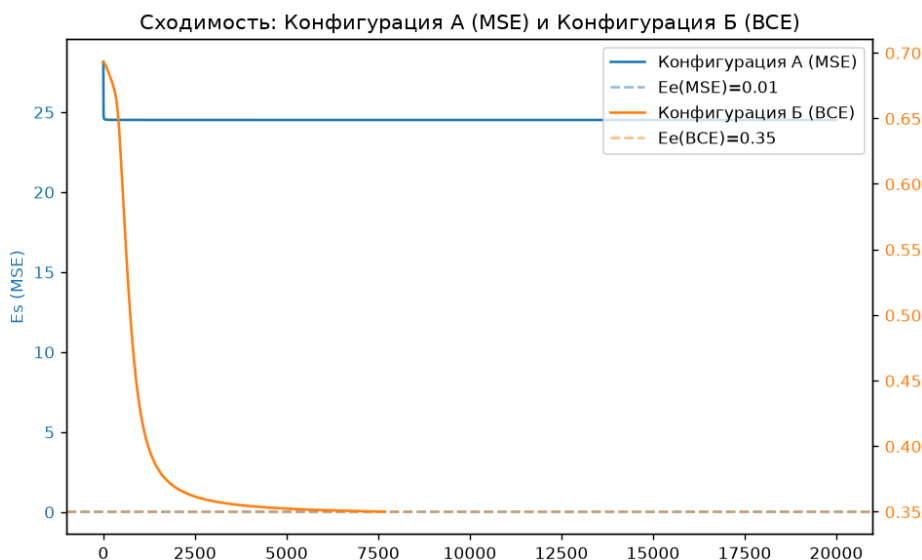
КОНФИГУРАЦИЯ А (MSE, без нормализации, $E_e = 0.01$)

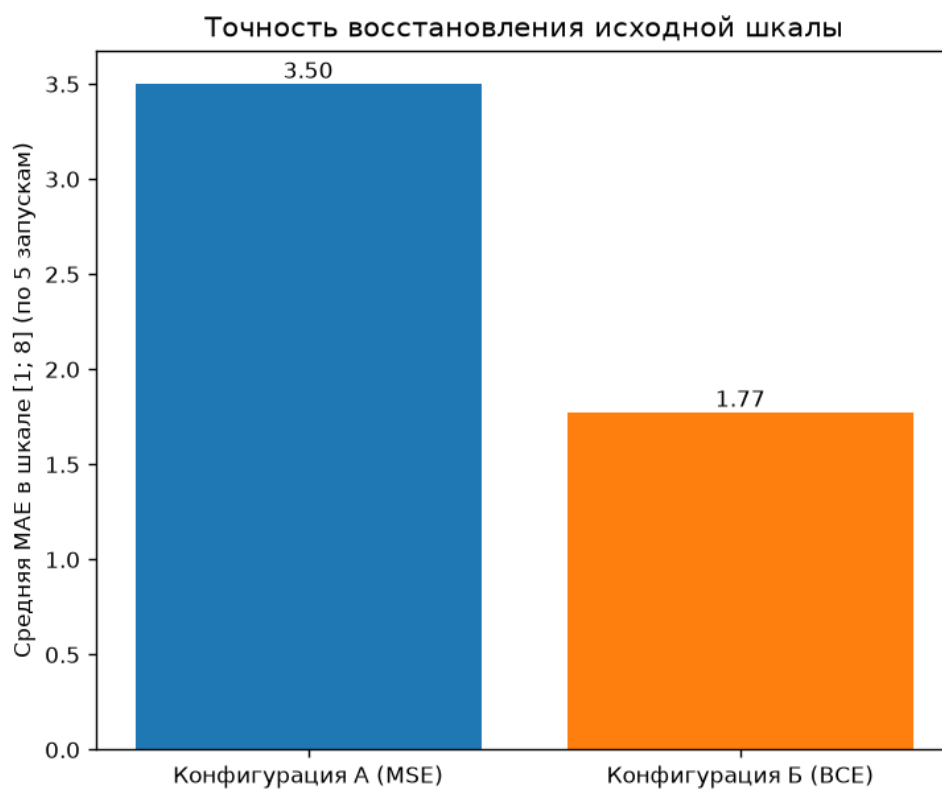
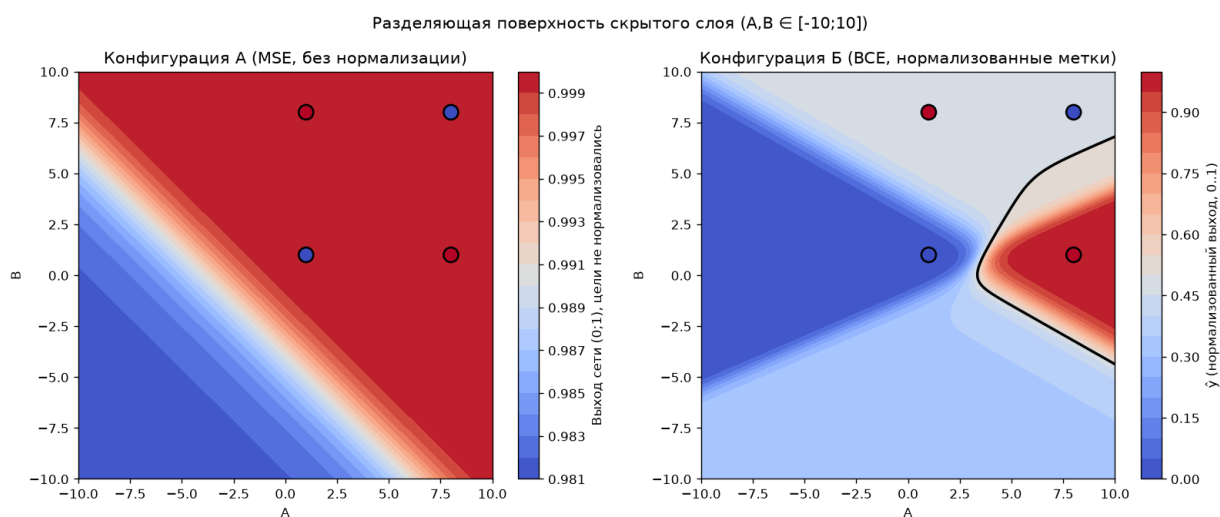
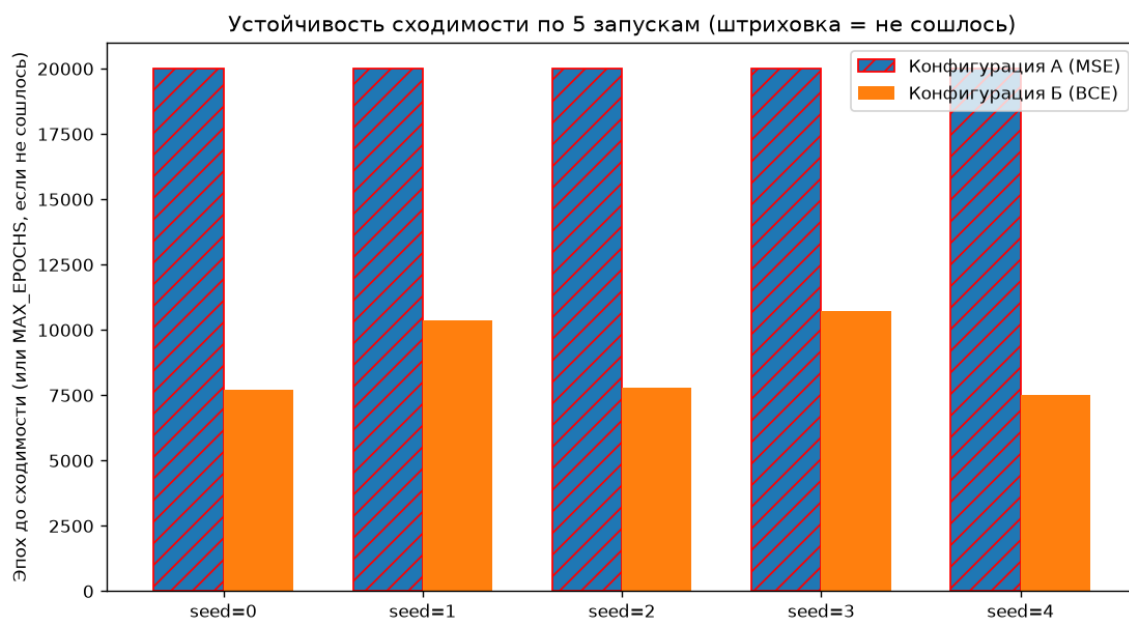
seed=0: НЕ СОШЛАСЬ за 20000 эпох, $Es_final=24.5001$, accuracy=0.50, MAE=3.50
 seed=1: НЕ СОШЛАСЬ за 20000 эпох, $Es_final=24.5001$, accuracy=0.50, MAE=3.50
 seed=2: НЕ СОШЛАСЬ за 20000 эпох, $Es_final=24.5001$, accuracy=0.50, MAE=3.50
 seed=3: НЕ СОШЛАСЬ за 20000 эпох, $Es_final=24.5001$, accuracy=0.50, MAE=3.50
 seed=4: НЕ СОШЛАСЬ за 20000 эпох, $Es_final=24.5001$, accuracy=0.50, MAE=3.50

КОНФИГУРАЦИЯ Б (BCE, нормализованные метки 0/1, $E_e = 0.35$)

seed=0: эпоха 7684, $Es_final=0.3500$, accuracy=0.75, MAE=1.77
 seed=1: эпоха 10340, $Es_final=0.3500$, accuracy=0.75, MAE=1.77
 seed=2: эпоха 7793, $Es_final=0.3500$, accuracy=0.75, MAE=1.77
 seed=3: эпоха 10721, $Es_final=0.3500$, accuracy=0.75, MAE=1.77
 seed=4: эпоха 7490, $Es_final=0.3500$, accuracy=0.75, MAE=1.77

Блок визуализации:





Результаты работы обученной сети в режиме функционирования:

Конфигурация А (MSE): сошлось 0 из 5
Ни один запуск не достиг критерия остановки.

Конфигурация Б (BCE): сошлось 5 из 5
Эпох до сходимости: min=7490, max=10721, среднее=8805.6, разброс=3231

РЕЖИМ ФУНКЦИОНИРОВАНИЯ СЕТИ (конфигурация Б)

--- Все 4 примера обучающей таблицы ---

Вход: (1, 1) -> $\hat{y}_{\text{norm}}=0.0072$, $\hat{y}_{\text{scaled}}=1.05$, класс≈1

Вход: (1, 8) -> $\hat{y}_{\text{norm}}=0.4992$, $\hat{y}_{\text{scaled}}=4.49$, класс≈1

Вход: (8, 1) -> $\hat{y}_{\text{norm}}=0.9942$, $\hat{y}_{\text{scaled}}=7.96$, класс≈8

Вход: (8, 8) -> $\hat{y}_{\text{norm}}=0.4996$, $\hat{y}_{\text{scaled}}=4.50$, класс≈1

--- Дополнительные пары (вне обучающей выборки) ---

Вход: (3, 3) -> $\hat{y}_{\text{norm}}=0.4127$, $\hat{y}_{\text{scaled}}=3.89$, класс≈1

Вход: (2, 7) -> $\hat{y}_{\text{norm}}=0.4989$, $\hat{y}_{\text{scaled}}=4.49$, класс≈1

Вход: (5, 5) -> $\hat{y}_{\text{norm}}=0.4988$, $\hat{y}_{\text{scaled}}=4.49$, класс≈1

Вход: (-4, 6) -> $\hat{y}_{\text{norm}}=0.2059$, $\hat{y}_{\text{scaled}}=2.44$, класс≈1

Вывод:

1. Конфигурация А (MSE) **не сошлась ни разу**, потому что сигмоида физически ограничена диапазоном (0;1) и не может приблизить выход к значению 8. Итоговая ошибка стабильно остаётся около 24.5, что делает критерий $E_e = 0.01$ недостижимым. Конфигурация Б (BCE) при нормализованных метках и пороге $E_e = 0.35$ **сходится во всех пяти запусках**, достигая ошибки 0.3500 за 7.5–10.7 тысяч эпох.
2. MSE даёт лишь 50% точности и $MAE \approx 3.50$, поскольку сеть фактически пытается регрессировать в недостижимую шкалу. BCE достигает 75% точности и $MAE \approx 1.77$ — почти вдвое лучше — благодаря корректной постановке задачи классификации и стабильному градиенту вида $(\hat{y} - \text{target})$.
3. Использование исходных меток 1 и 8 приводит к насыщению сигмоиды и исчезновению градиента, что делает обучение невозможным. После нормализации в 0/1 задача становится согласованной с функцией активации, и сеть начинает обучаться корректно. Линейный пересчёт $\hat{y}$ обратно в шкалу [1;8] не ухудшает качество.
4. MSE показывает полную устойчивость в плохом смысле: все пять запусков дают одинаково плохой результат, что указывает на систематическую проблему, а не на влияние seed. BCE, напротив, стабильно достигает критерия остановки во всех пяти запусках, хотя число эпох заметно варьируется — от 7490 до 10721.
5. Задача остаётся XOR-подобной вне зависимости от того, используем ли мы метки 0/1 или 1/8. Точки расположены крест-накрест, и разделить их одной прямой невозможно. Шкала меток влияет только на устойчивость обучения, но не на саму структуру задачи.
6. Функция потерь не заменяет необходимость скрытого слоя. И MSE, и BCE применяются к одной и той же архитектуре 2-2-1; различие между ними влияет на скорость и качество обучения, но не делает однослойную сеть способной решить XOR. Нелинейность обеспечивается скрытым слоем, а не типом ошибки.